MAP OF PROJECT AETHER

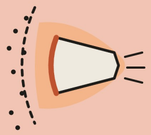
GPU systems engineering for thermochemical hypersonic environment reconstruction · June 5 – August 25, 2023

THE METHOD — Every gap between the legacy CPU code and Hopper was a data-movement problem before it was a compute problem. Profile the bytes, then pick the fusion, format or algorithm that deletes them.

NEXT FRONTIER — Coupled multi-fidelity runs: the surrogate and the particle solver iterating together in the loop.

① THE PROBLEM

Hypersonic re-entry at altitude: the mean free path approaches vehicle scale, the continuum model breaks, and the gas has to be simulated as particles.



DSMC (BIRD) · EVERY TIMESTEP

MOVE → BIN BY CELL → COLLIDE

▲ the bottleneck

- **SORT / BIN EVERY STEP** — irregular, memory-bound, the majority of timestep time
- **RADIATION + ABLATION TALLIES** — scatter updates that defeat cache reuse
- **DIGITAL-TWIN SURROGATE** — a latency-bound serving problem, not batch

LEGACY · 32-CORE CPU	HOPPER · H100
O(10–100) heavyweight threads	O(100k) threads, 1 particle each
~100 GB/s per socket, shared	~3 TB/s HBM3
implicit per-core caches	228 KB/SM explicit shared memory
stateful Mersenne Twister	counter-based Philox / Threefry

Week 1–2: roofline + trace analysis → signed-off baseline and a bottleneck map

② FUSED TRITON SORT

The naive GPU port ran binning as four kernels. Every kernel boundary was a full round trip of particle state through HBM.

NAIVE PORT · 4 KERNELS

CELL ID → HISTOGRAM → GLOBAL SCAN → SCATTER

each arrow = full state round trip to HBM

FUSE

ONE TRITON KERNEL · WHOLE TILE RESIDENT IN SRAM

CELL IDS	HISTOGRAM	LOOK-BACK	SCATTER
in registers	block-local, SRAM	Onesweep-style	from SRAM offsets

+ Philox RNG folded in: a few ALU ops per particle, zero RNG state in memory
+ fused epilogue: scatter + cell-density stats for the collision stage
+ Structure-of-Arrays: the sort moves keys + indices, never payloads
+ tiles autotuned to fit 228 KB SMEM/SM — utilization over occupancy

WHERE THE 50× CAME FROM (FACTORS MULTIPLY)

6–8×	3–4×	~2×
fusion + SRAM residency (no HBM round trips)	single-pass digit strategy	fused RNG + epilogue

50× particles/s vs 32-core CPU

4M particles · 10k cells · collision stats match CPU within noise

③ COMPILER FUSION + TMA

The whole timestep is one jitted JAX function, so XLA sees it globally and fuses what no per-kernel engineer would. HLO dumps audited stage by stage.

JAX TRACE → HLO GRAPH → XLA FUSION → FUSED KERNELS

STATE-ARRAY PASSES PER TIMESTEP · UNFUSED → FUSED

Collision search + binning	□□□ →	■ into the sort
Force + energy accumulation	□□□ →	■
Velocity / position integration	□□ →	■
Boundary + ablation tally	□□ →	0 · in registers
RNG draw + scattering update	□□ →	■

3.6× fewer HBM bytes

TOTAL 12 → 3.3 passes (measured) · memory-bound, so ≈ wall-clock

HOPPER TMA · CUDA C++

For the irregular stages XLA can't fuse: **cuda::memcpy_async** bulk-tensor copies stage the next tile while this one computes; **mbarriers** track arrival, so no registers burn on in-flight addresses.

TMA	load tile 0	load tile 1	load tile 2	load tile 3
SM MATH		math tile 0	math tile 1	math tile 2

copy latency hidden behind compute · freed registers raise occupancy · CuTe (CUTLASS 3.0) idioms

④ PHYSICS SURROGATE

The digital twin needs flow fields orders of magnitude faster than DSMC — without leaving the physics behind.

PINN LOSS
data error + PDE-residual penalty

↓

GALERKIN / FOURIER ATTENTION
softmax-free, linear in sequence length

↓

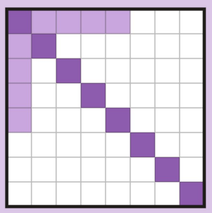
75% BLOCK-SPARSE MASK
local windows + global tokens

↓

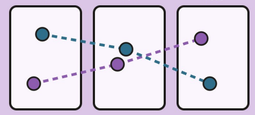
FLASHATTENTION-2 TILING
softmax statistics never leave SRAM

↓

MOTR TRACK QUERIES
"follow this parcel" = one decode step



illustrative block mask · dark = local window, light = global · 16 of 64 tiles computed



track queries keep particle identity across timesteps

FNO-style spectral mixing over the spatial field. Linear attention is what lets the sparse + FP8 kernels of ⑤ reach the attention stage.

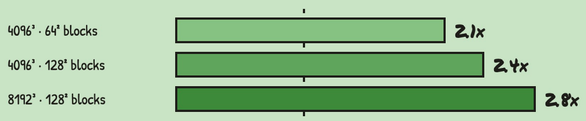
⑤ SPARSE + FP8 NUMERICS

Shrink the arithmetic itself: skip three of every four blocks, then halve every weight.

CUTLASS 3.X BLOCK-SPARSE GEMM

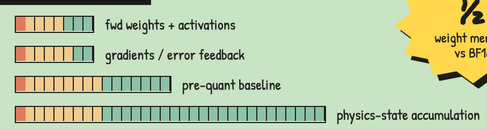
CuTe collective builder → warp-specialized Hopper mainloop (TMA loads + WGMMA math); sparse variants from the same templates via a compile-time block mask.

2–3× vs dense cuBLAS



H100 SXM · BF16 compute · 75% block sparsity · dashed line = dense cuBLAS (1×)

FP8 QUANTIZATION



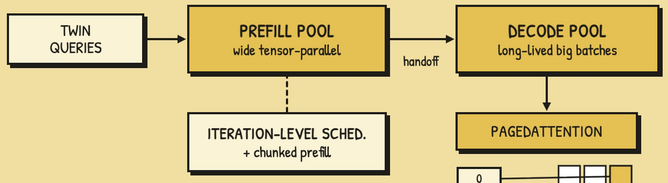
½ weight memory vs BF16

■ sign ■ exponent ■ mantissa · FP16 skipped (range-limited)

post-training quantization, per-tensor scales + light calibration; stayed floating-point over INT8

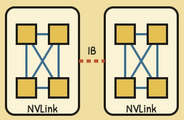
⑥ DISAGGREGATED SERVING

Prefill is compute-bound and parallelizes wide; decode is bandwidth-bound and wants big batches. One shared pool serves both badly.



block tables over fixed pages: <4% KV waste vs 60–80% contiguous; a what-if branch copies the table, not the state

SCCL-SYNTHESIZED COLLECTIVES



All-reduce + all-to-all solved for our NVLink/IB topology; TACCL-style sketches prune the search, ≈40% less collective time on the decode critical path. Megatron TP inside a node, GSPMD sharding across nodes.

3× placement + scheduling × **1.7×** paged-cache batches × **1.8×** synthesized collectives = **9×** lower latency

end-to-end, at matched throughput, vs monolithic serving + stock NCCL

WHERE THE BYTES MOVE

↑ fast · tiny · close to the ALU

REGISTERS — cell ids, Philox draws, ablation tallies — ② ③

SHARED MEM — 228 KB/SM, explicit — sort tiles · TMA tiles · softmax stats — ② ③ ④

TENSOR CORES — WGMMA · FP8 E4M3/E5M2 · block-sparse tiles — ⑤

HBM3 · ~3 TB/s — 12 → 3.3 state passes; FP8 halves decode weight fetch — ③ ⑤

NVLINK / IB — pool-to-pool handoff · synthesized all-reduce + all-to-all — ⑥

↓ slow · vast · across the cluster

Numbered dots = the workstream that attacked that level. Every win on this map is a byte that stopped moving.

THE 12-WEEK EXPEDITION

Phases overlap on purpose — fusion shared the kernel work's profiling infrastructure, and serving consumes the sparse / FP8 GEMMs. Every exit gate closed inside the window.

WK 1 · WK 2 · WK 3 · WK 4 · WK 5 · WK 6 · WK 7 · WK 8 · WK 9 · WK 10 · WK 11 · WK 12

- **1 · BASELINE + PROFILING** · Jun 5–16 — gate: signed-off baseline + bottleneck map
- **2 · TRITON KERNELS** · Jun 19 – Jul 7 · radix sort, SRAM tiling, Philox, autotune — gate: 50× particles/s vs CPU baseline ✓
- **3 · COMPILER FUSION + TMA** · Jul 3–21 · XLA timestep fusion, mbarrier pipelines — gate: 3–4× fewer bytes to HBM per timestep ✓
- **4 · SPARSE NUMERICS + FP8** · Jul 17 – Aug 4 · CUTLASS, E4M3/E5M2 calibration — gate: 2–3× over cuBLAS at 75% sparsity; −50% weight memory ✓
- **5 · DISAGGREGATED SERVING** · Aug 7–25 · prefill/decode pools, paged cache, SCCL synthesis — gate: 9× lower end-to-end latency + final report ✓

④ surrogate work threads through phases 3–5: its sparse mask, FP8 weights and track queries are what the numerics and serving layers run.

Every figure is a measured result from the AETHER benchmark suite (4M-particle DSMC case; H100 SXM), correctness-validated before the speed claim was accepted.